

Monoalphabetic Substitution Cipher and its Cryptanalysis Using Frequency and Pattern Analysis (Except python any language can be used)

- Implement the Monoalphabetic substitution cipher and generate the ciphertext. The plaintext (at least 1 page) should be selected from the Book “Modern Cryptography” by Katz and Lindell. Each group can use text written on page number (Group_number + 30) page.
- Perform **letter-frequency analysis** on ciphertext
 - Counts the frequency of each ciphertext letter
 - Displays the letters in descending frequency order
 - Calculates the percentage frequency of each letter
 - Identifies the most frequent letters
- Use **word patterns and repeated words** to generate possible substitution
 - One-letter words
 - Two-letter words
 - Three-letter words
 - Repeated words
 - Repeated letter patterns
- Iteratively recover plaintext from ciphertext (Don’t use any library function)
 - Propose candidate substitutions
 - Replace suspected letters
 - Examine the resulting partial plaintext
 - Reject incorrect hypotheses
 - Continue until meaningful plaintext is obtained
- Recover the corresponding substitution key
- Validate the recovered key by re-encrypting the plaintext
- Document and justify their cryptanalytic decisions using the table given below

[\(NoteBook\)](#)

Your program should contain at least the following functions/modules:

```
1. frequency_analysis()
2. word_frequency_analysis()
3. pattern_analysis()
4. apply_substitution()
5. display_partial_plaintext()
6. verify_solution()
```

Step	Observation	Possible Substitution	Substitution Tested	Result	Decision
1	Q occurs most frequently	$Q \rightarrow E$	$Q \rightarrow E$	Many substitutions were useful	Good decision

*its an example table, fill the entries as per your experiment and understanding **Write the algorithm in the notebook and add code to your existing repository**